# CE310: Evolutionary Computation and Genetic Programming

Coursework Assignment, Prof Riccardo Poli, Spring Term 2025-26

Table of Contents


1 Overview ..... 2
1.1 What needs to be done in the coursework ..... 2
1.2 How you should develop your coursework ..... 2
1.3 Format ..... 3
1.4 Submission ..... 3
1.5 Clarifications and Recommendation ..... 3
2 GA implementation and solving problems (Part 1) ..... 4
2.1 Task 1 - Implement a Binary Generational GA ..... 4
2.2 Task 2 – Use your GA to Evolve Sequences of Integers ..... 4
2.3 Task 3 – Tune the GA and solve 15-max and soft-15-max problems ..... 5
2.4 Task 4 – Invent a deceptive problem and compare with 15-max ..... 5
3 GP implementation and solving problems (Part 2) ..... 6
3.1 Task 5 – Representation and Genetic Operators ..... 6
3.2 Task 6 – Program Execution and Fitness Functions ..... 6
3.3 Task 7 – Experiments and Analysis of Results ..... 9
3.4 Task 8 – Evolution at the Level of Primitives ..... 9
4 Marking Scheme ..... 10
4.1 Incremental Development & Methodology ..... 10
4.2 Responsible AI Use ..... 10
4.3 Task 1 Binary GA (10%) ..... 11
4.4 Task 2 Genotype–Phenotype Mapping & 15-max (10%) ..... 11
4.5 Task 3 Parameter Tuning & Experiments with 15-max and soft-15-max (15%) 11
4.6 Task 4 Deceptive problem and 15-max Comparison (15%) ..... 11
4.7 Task 5 GP Representation & Operators (10%) ..... 11
4.8 Task 6 Interpreter & Fitness Functions (20%) ..... 12
4.9 Task 7 Experimental Protocol & Statistics (10%) ..... 12
4.10 Task 8 Evolution of Primitives (10%) ..... 12

# 1 Overview

## 1.1 What needs to be done in the coursework

This coursework is worth 40% of the module overall (the other 60% being the exam). It consists of two parts, each carrying 50% of the marks for the coursework.

Part 1 involves **coding your own Genetic Algorithm** in Python and using it to **solve a small number of problems**, documenting and discussing your work along the way.

Part 2 involves turning the Genetic Algorithm developed in Part 1 into a **Stack-based Genetic Programming system** and using it to **solve a small number of problems**, recording the run behaviour and performance, and discussing the results obtained.

The **deadline** for submitting the coursework on FASER is **2026-03-06 (Week 23) just before 2pm.**

## 1.2 How you should develop your coursework

You are **allowed to use AIs to help with some aspects of the coursework**, but you do not have to do so. If you do, you must **use AIs responsibly and prove that you did so**.

Whether or not you use AIs to help, you must **make sure you fully understand the work you are submitting** to the point that you could satisfactorily answer any questions I may have at the time of marking on the code or analysis done.[1]

To promote this level of understanding, I am making it **a requirement that you produce the coursework <u>incrementally</u> and you provide evidence that you did so**. Basically, you need to follow the same *<u>small steps</u>* we have used in the CE310 labs/classes, the only difference being that, in your coursework, you need to **describe and justify the sequence of small steps that eventually lead to implementing algorithms, solving problems,** etc., while in the classes I provided this information.

If you make use of AIs, you need to **show the exact queries you submitted to the AIs and the AI outputs**. You must also **show evidence that you challenged the AIs**, indicating how you identified logical errors and debugged the AI ideas and code.[2]

Whether or not you use AIs to help, **if you make use of concepts and techniques that were not presented in CE310, you must justify why you chose them** over the alternatives that I presented.

---

[1] Be aware that I will myself feed this coursework description in multiple AIs (multiple times) and will record their outputs. So, if you decide to just do this to save time and effort, I will most likely find out. If I do, I don't consider this a responsible use of AI.

[2] If you manually corrected an AI's outputs, you need to *resubmit such corrections into the AI*, so it is aware of where you stand, in case of further interactions. Type into the AI prompt something like "*Below is my current code. I don't want any answer. Just making you aware of changes I made: ** YOUR CODE***".

AIs tend to produce the same or similar answers repeatedly and they write code in a very characteristic style. So, *your coursework submission may be flagged up by our in-house and commercial plagiarism detection systems*. However, **you will not be penalised for this, if your coursework shows clearly that you used the AIs responsibly**.

To reiterate: If you want to get help from AIs in producing the coursework, you must **keep a log of the AI interactions performed and to submit it with the coursework**. This should be in the form of a Word file where you copy and paste all AI interactions you have had in the order you had them.

## 1.3 Format

The required format for the coursework is a **single Jupyter notebook for each part** (that is: submit two notebooks), **plus the AI log if you used AIs**.

In each notebook you will have *multiple cells* (text/markup, code and output). As I indicated before, **you must proceed in very small steps**, like we do in the labs. Follow this *protocol*:

1. **Precede each code cell with a textual explanation** of **what** you want to achieve with the code that immediately follows and **why** it is the next logical step towards the goal.
2. The code should just implement the target functionality, but it must also **include tests** and produce outputs, so one is sure it works as expected. If the code presents bugs, try to fix any issues yourself (preferred) or ask the AI for help.
3. **Make sure the code does not only run, but you fully understand it**. If you don't, study it more, add more tests, or *split the task into even smaller steps*, until you do understand it all.
4. Then and only then, **identify the next (smallest) element of functionality** that makes sense to add, **and repeat** the steps above, until you achieve the target.

## 1.4 Submission

Please submit all your work as a **single zip file** on FASER by the above deadline. Inside the zip file, put:

1. a Jupyter notebook with your Part 1 work,
2. a Jupyter notebook with your Part 2 work,
3. if AIs were used, also include the AI Log in the form of a Word document.

## 1.5 Clarifications and Recommendation

Ask questions during the lectures and classes if any requirement in the assignment is not clear. Feel free to also contact me by email.
You can assume that if you struggle, your fellow students will also struggle. Keep in mind that there are many ways how to address and complete the tasks. There is no single correct or best way to do this.

Above all, **start this process early(!)**, so that you have time to try things and make mistakes, and have still have time to learn from them.

# 2 GA implementation and solving problems (Part 1)

## 2.1 Task 1 - Implement a Binary Generational GA

Incrementally develop, following the protocol mentioned above, **a binary <u>generational</u> Genetic Algorithm.**

Your *generational* GA will need:

1. initialisation,
2. fitness evaluation (more on this below),
3. tournament selection (with any tournament size $T$ from 2 upward),
4. cloning,
5. mutation (used as an independent genetic operator, not on top of other genetic operators), and
6. crossover.

All the above will need to be tested independently and integrated gradually within the GA's **main loop**.

Use the one-max function to test the GA.

Choose **meaningful settings** for: the tournament size $T$, the cloning probability, the crossover probability and the two mutation probabilities: the probability of applying the mutation operator and the probability of mutating each bit.

Verify or revise/tweak your choices using multiple runs of the GA.

## 2.2 Task 2 – Use your GA to Evolve Sequences of Integers

Now we need to use the binary GA to evolve sequences of $L$ integers, each in the range 0 to 15 for a pre-assigned $L$.

The chromosomes in your GA will still be binary. However, before fitness evaluation groups of contiguous bits are transformed into integers. This operation is a *genotype-to-phenotype transformation*, where genotypes are bit strings/chromosomes, and phenotypes are sequence of $L$ integers.

**Important**: two different forms of conversion *from bits to integers must be explored:*

1. the standard binary to decimal formula where each group of 4 contiguous bits $(b_1, b_2, b_3, b_4)$ is transformed into integers as follows:

   $$I = b_1 + 2 \times b_2 + 4 \times b_3 + 8 \times b_4$$

   So, to evolve sequences of $L$ integers one needs chromosomes of $4 \times L$ bits.

2. the less usual *non-positional* binary to decimal conversion where each integer is encoded using a group of 15 contiguous bits $(b_1, b_2, \ldots, b_{15})$ *which are transformed into integers as follows:*

$$I = b_1 + b_2 + \ldots + b_{15}$$

So, to evolve sequences of $L$ integers one needs chromosomes of $15 \times L$ bits.

To test everything is working, implement a **15-max fitness function**, which is like one-max but:

1. it first applies the appropriate genotype-to-phenotype conversion to get a sequence of $L$ integers and then
2. returns how many of them equal 15

So, 15-max fitness values range between 0 and $L$ inclusive.

**Run the GA**, with both binary representations for integers, to ensure all is working as expected.

## 2.3 Task 3 – Tune the GA and solve 15-max and soft-15-max problems

Perform runs, each lasting 50 generations, to try to identify the **good values of population size and tournament size**.

With the good parameters found (and still 50 generations), for each representation for integers and for both $L = 10$ and $L = 30$ perform **10 independent runs** (recording the best-of-run best fitness found in each) with the **15-max problem**.

Modify the 15-max fitness function, call it *soft-15-max*, so that instead of counting how many integers are exactly 15, **it returns the average of the $L$ integers**.

Repeat the experiments **to find out whether this fitness function helps or hinders** the ability of the GA to solve the 15-max problem.

## 2.4 Task 4 – Invent a deceptive problem and compare with 15-max

Based on the binary deceptive fitness function we have studied in the Unit 03 class (which is known in the GA literatures as *trap function*), **define a deceptive version of the 15-max problem.** Rerun your code to prove that the GA ability to solve the problem is slowed down and sometimes the population converges towards the deceptive local optimum.[3]

[3] Feel free to modify either the 15-max or soft-15-max functions to create your 15-trap function. Just make sure you compare the results obtained with your *trap* function with the corresponding version of the 15-max function.

# 3 GP implementation and solving problems (Part 2)

Part 2 requires transforming one of the GAs developed in Part 1 into a GP system. This will require changes to some, but not all, the elements of the GA.

## 3.1 Task 5 – Representation and Genetic Operators

Pick the GA which you feel worked best in Part 1.

Assume that our GP primitive set only includes a very small number of instructions:

- NOP: that does nothing,
- +: which adds up two values and returns their sum,
- -: which subtracts two values and returns their difference,
- *: which multiplies two values and returns their product,
- X: which returns the input value to the program,
- 1: which returns the constant 1,
- -1: which returns the constant -1, and
- 0: which returns the constant 0.

So, in total we have 8 instructions.

Instructions will be represented as integers (NOP=0, +=1, -=2, etc.). **Programs are just sequences of $L = 30$ integers**, represented using bit strings of length $L \times 3$ or $L \times 7$ bits depending on which formula you use to decode integers:

$$I = b_1 + 2 \times b_2 + 4 \times b_3 \qquad \text{OR} \qquad I = b_1 + b_2 + \ldots + b_7$$

Because we are evolving programs represented as binary strings, in our GP system you are asked to reuse the **initialisation, selection, crossover, mutation and main evolution loop used for the binary GA** in Part 1.

## 3.2 Task 6 – Program Execution and Fitness Functions

We know that in GP, computing the fitness of a program often requires running the program multiple times with different inputs. The question is: *how are the instructions sequenced and how do they know on which data to operate and where to place their results?*

We are used to seeing programs/expressions like: (X + 1) or (X * X – 1). This is called the **infix notation** as operations are *in between* their arguments (e.g., + is between X and 1 in the first expression). In lectures we will see the **prefix notation** where (X + 1) would be written as (+ X 1). *In this assignment we will use another notation called the* **postfix notation** (a form of Reverse Polish Notation) where the operations follow their arguments so (X + 1) would be written as (X 1 +). We do this as it leads to an extremely simple simulator of program execution.

The operations are executed using the following **simulator/interpreter** which you need to incorporate, test and understand thoroughly, and, finally, use in your GP code:[4]

```
# Simulator of program execution (GP interpreter)
# program = sequence of integers
# x = input variable
def execute(program, x):

    # A place where to get arguments for operations and
    # for storing their results
    stack = []

    # If l = [1,2,3], l.pop() removes 3 from the list and returns it
    # but if l = [], l.pop() gives an error
    # Fixing the problem for stack by defining our own pop()
    def pop():
        if not stack:
            return 0
        return stack.pop()

    # Main execution loop
    for instr in program:
        if instr == 0:   # NOP
            pass
        elif instr == 1: # X
            stack.append(x)
        elif instr == 2: # +
            stack.append(pop() + pop())
        elif instr == 3: # -
            stack.append(pop() - pop())
        elif instr == 4: # *
            stack.append(pop() * pop())
        elif instr == 5: # 1
            stack.append(1)
        elif instr == 6: # -1
            stack.append(-1)
        else:            # 0
            stack.append(0)

    # Returning the very last result appended to stack
    return pop()
```

So, from the point of view of this simulator/interpreter the individual `[5, 1, 2]` corresponds to `[1, X, ADD]`, that is `(1 X +)` in postfix notation.

So, running:

```
execute([5, 1, 2], 3)
```

[4] The simulator presents some resemblance with the CPU simulator presented in the Unit 05 class. However, in that simulator instructions made use of registers, while here the instructions use a stack.

performs the following steps:

```
Input x = 3

Initial State:
      Stack: []

Step 1: Instruction 5
      [Append] 1
      Stack: [1]

Step 2: Instruction 1
      [Append] x (3)
      Stack: [1, 3]

Step 3: Instruction 2
      [Pop]  -> 3
      [Pop]  -> 1
      [Calc] 1 + 3 = 4
      Stack: [4]

Return instruction:
      [Pop]  -> 4

Final Result: 4
```

Turning to **fitness functions**: the details of your fitness functions will depend on the *problem* you need to solve, but they all will most certainly *require invoking the function* **`execute(prog, x)`** *for multiple fitness cases*, accumulating the contributions to fitness from each case, and finally returning the accumulated fitness.

The problems you will need to solve are:

1. **Problem 1**: Evolve programs that when executed with x = -1 produce the **smallest possible output** (i.e., a value as negative as possible) and when executed with x = 1 produce the **largest possible output** (i.e., a value as positive as possible).

2. **Problem 2**: Evolve a program that when executed 21 times with $x$ = -1, -0.9, ... , 1.0 produces outputs that are as close as possible to the $y$ values obtained from the function (5th order polynomial):

$$y = 2 \times x^5 + 4 \times x^4 + 6 \times x^3 + 8 \times x^2 + 10 \times x + 1$$

You need to **develop corresponding fitness functions for these two problems**, making sure they achieve the high-level goals specified in the problem statements above.

## 3.3 Task 7 – Experiments and Analysis of Results

You are asked to **perform a series of GP runs** and describe the results of your runs in a **report**. In your experiments you will need to use GP in different configurations (i.e., problems and parameters).

The aim of the experiments is to get an intuition *on how the population and tournament size impacts on fitness of the evolved programs* for different symbolic regression problems, and on the corresponding *computational effort* (based on how often the fitness function is being executed).

Since GP is a stochastic searcher, you will see that performance varies from run to run. Therefore, to draw your conclusions, you should ensure you **perform at least 10 runs in each experiment**. *IMPORTANT: Do not stop runs even if an ideal solution has been identified. So, they only stopping criterion for runs should be that the number of generations has reached the limit (50 generations).*

Once the runs are completed, **analyse the output and behaviour** of the system produced in different experiments. To do that you will need to look at statistics obtained in different experiments when solving the chosen problems. Statistics often used to describe individual GP runs include

- The average and standard deviation of the fitness of the programs in the population
- The fitness of the best program in the population

Normally these are computed at each generation.

Because you have 10 runs in each experiment, you need to **aggregate results** across runs. Typical **averaging** is sufficient for this purpose, but you should keep an eye also on variability of results across runs with the same configuration, so reporting also the **standard deviation** would be good.

When performing multiple runs, it is also important to look at:

- The fraction of runs where an ideal solution was found
- The average generation at which such solution was found (given that we are not stopping runs until they reach the final generation).

Study the results and report any conclusions.

## 3.4 Task 8 – Evolution at the Level of Primitives

After random initialisation, we expect to see each of our 8 primitives to be present in approximately equal proportions (12.5%) across all chromosomes in the population. However, the utility of the primitives may change generation after generation (in ways that depend also on the problem being tackled), and so their proportions in the population will not be equal anymore.

Study these changes (in at least the final generation, or in all generations) and try to formulate an explanation for why certain primitives are significant more frequent than

others. Why are such primitives spreading within the population? Why are they fitter than others?

# 4 Marking Scheme

## 4.1 Incremental Development & Methodology

In both Part 1 and Part 2, I compliance with the required *lab-style incremental development* is required. Marks in each part can be reduced by up to 20% if the this has not been followed.

**No reduction of marks will be associated with:**

- Each code cell is preceded by a clear explanation of what functionality is being added and why it is the next logical step.
- Functionality developed in small, testable steps.
- Each component (initialisation, selection, mutation, crossover, fitness, loop) tested independently before integration.
- Evidence of debugging and refinement.
- Clear justification of parameter choices.

**Medium reduction of marks will be associated with:**

- Mostly incremental, but steps too large or poorly justified.
- Some but limited testing evidence.
- Weak justification of choices.

**High reduction of marks with be associated with:**

- Large blocks of code with minimal explanation.
- Little evidence of stepwise reasoning or understanding.

## 4.2 Responsible AI Use

Marks in each part may be reduced by up to 20%, or an investigation into an academic offence[5] can be initiated, based on the **degree to which AI has been used irresponsibly** (as described in the main text.

**No marks will be deducted AI not used or if:**

- Full AI log submitted
- Evidence of critical engagement
- Evidence of debugging/challenging the AI output

[5] See https://www.essex.ac.uk/student/academic-offences/artificial-intelligence.

## 4.3 Task 1 Binary GA (10%)

Marks awarded for correct implementation of:

- Initialisation
- Tournament selection
- Cloning
- Mutation (operator-level + bit-level probabilities)
- Crossover
- Fully working generational loop
- Testing on one-max

## 4.4 Task 2 Genotype–Phenotype Mapping & 15-max (10%)

Marks awarded for:

- Both binary encodings implemented
- Correct 15-max fitness function
- Proper testing

## 4.5 Task 3 Parameter Tuning & Experiments with 15-max and soft-15-max (15%)

Marks awarded for:

- Systematic parameter optimisation or non-systematic but good reasoned choices
- 10 independent runs per condition
- Correct implementation of soft-15-max
- Experiments repeated properly
- Appropriate recording, presentation and understating of results
- Clear comparison and explanation

## 4.6 Task 4 Deceptive problem and 15-max Comparison (15%)

Marks awarded for:

- Correct trap-style function
- Demonstrated slowdown/local optima
- Clear explanation

## 4.7 Task 5 GP Representation & Operators (10%)

Marks awarded for:

- Correct reuse of GA framework
- Correct primitive encoding
- Working operators

## 4.8 Task 6 Interpreter & Fitness Functions (20%)

Marks awarded for:

- Correct use of provided interpreter
- Fitness functions correctly defined
- Proper aggregation across fitness cases
- Clear explanation of objectives

## 4.9 Task 7 Experimental Protocol & Statistics (10%)

Marks awarded for:

- 10 runs per experiment
- 50 generations per run (no early stopping)
- Per-generation statistics
- Aggregated statistics across runs
- Ideal-solution reporting
- Clear interpretation of results
- Link to EC theory
- Discussion of parameter effects
- Discussion of variability

## 4.10 Task 8 Evolution of Primitives (10%)

Marks awarded for:

- Primitive frequency tracking
- Clear explanation of trends